Think with Images, Learn from Failure: Training-Free Self-Evolution of Visual Agents via Skill and Tool Generation

Anonymous Author 1[1], **Anonymous Author 2**[1] **and Anonymous Author 3**[1]
[1]Anonymous Institution

[*]Work done during an internship at Alibaba. [†]Corresponding author.

Visual language models still exhibit systematic failure patterns on standard benchmarks, yet closing these gaps currently demands either RL training on dedicated GPU clusters or manual prompt engineering. We present EvoAgent, a training-free framework in which VLM agents autonomously evolve two complementary capability types from failure: *skills*—structured reasoning strategies encoding how to approach a problem class—and *tools*—executable Python programs that expand the agent's visual perception by manipulating images directly. On each failure, EvoAgent visually diagnoses the root cause by inspecting both the original image and any intermediate tool outputs, decides whether the bottleneck is cognitive or perceptual, generates the appropriate capability, validates it, and accumulates it for future cases—all without any weight updates. Evaluated on four standard VQA benchmarks (ChartQA, V*, HRBench, MathVista), EvoAgent achieves improvements of +2.8%–9.2% on held-out validation sets after only three evolution iterations on 200 training cases per benchmark. Our central finding is that skill evolution is universally effective (4/4 benchmarks) while tool evolution provides additional gains only on perceptually challenging benchmarks (2/4), revealing a fundamental dichotomy: most systematic VLM failures on standard benchmarks are *strategy* failures, not perception failures. These results show that failure-driven in-context evolution is a viable, cost-efficient alternative to training-based approaches for visual reasoning improvement.

1. Introduction

Large vision-language models (VLMs) have made remarkable strides on visual benchmarks, yet they still exhibit systematic, repeatable failure modes on tasks that require structured visual reasoning. A chart that demands multi-step data extraction, a high-resolution image that requires isolating a sub-region, a mathematical diagram that calls for compositional interpretation: these cases expose not random errors but consistent gaps in strategy and perception. Current approaches to closing these gaps fall into two camps, both expensive. Training-based methods such as Anonymous (2025e) and Anonymous (2025b) apply reinforcement learning or supervised fine-tuning on large GPU clusters to teach VLMs to use tools. Prompt-based methods such as chain-of-thought or few-shot exemplars improve performance on narrow distributions but do not generalise across benchmark families and require manual curation.

A third approach—letting the agent learn from its own mistakes—has shown promise in text-only domains (Shinn et al., 2023; Wang et al., 2024), but it stops short of what visual reasoning requires. Reflexion (Shinn et al., 2023) accumulates verbal reflections per case, yet these reflections are not persistent across cases, never become reusable capabilities, and cannot encode executable visual processing. No prior work achieves training-free generation of *both* reusable reasoning strategies *and* executable visual tools that generalise to unseen cases.

We present EvoAgent, a training-free framework in which a VLM agent autonomously evolves two complementary capability types from failure. **Skills** are structured reasoning strategies—standard operating procedures (SOPs) encoding *when* and *how* to approach a problem class. **Tools** are executable Python programs that expand the agent's visual perception by manipulating images directly. On each failure, a two-role agent

Alibaba Qwen Applications Business Group

Figure 1 | **EvoAgent evolution loop and system comparison.** *(Left)*: on each failure, the AnalyzerDecider visually inspects the image and any tool outputs, diagnoses the bottleneck type, and directs the Generator to produce a reasoning skill (SOP), an executable visual tool (Python), or both. Generated capabilities are validated and accumulated for future cases. *(Right)*: EvoAgent achieves competitive accuracy against RL-trained methods across ChartQA and V* at zero training cost (✗ = training required, ✓ = training-free).

(AnalyzerDecider + Generator) visually diagnoses the root cause by inspecting the original image alongside any tool outputs, decides whether the bottleneck is cognitive or perceptual, generates the appropriate capability, validates it, and accumulates it for future cases—all without updating model weights.

After only three evolution iterations on k=200 training cases per benchmark, EvoAgent achieves consistent improvements of +2.8%–9.2% on four standard VQA benchmarks (ChartQA, V*, HRBench, MathVista) evaluated on held-out validation sets, outperforming all training-free baselines including Reflexion (Shinn et al., 2023) and few-shot chain-of-thought. Our most consequential finding is that *skill evolution is universally effective* across all four benchmarks, while tool evolution provides additional gains only on benchmarks with perceptual bottlenecks—revealing that most systematic VLM failures are *strategy* failures, not perception failures.

Contributions.

(1) We propose EvoAgent, a training-free framework for VLM agents to autonomously evolve reasoning skills and visual tools from failure, achieving +2.8%–9.2% improvements on four VQA benchmarks without any weight updates.
(2) We identify a **cognitive-perceptual bottleneck dichotomy**: skill evolution alone suffices for strategy-bottlenecked benchmarks (ChartQA, MathVista), while perceptual benchmarks (V*, HRBench) additionally require tool evolution. This decomposition provides a principled diagnostic for where to invest improvement effort for a given benchmark.
(3) We introduce **progressive tool mastery**: after generating a tool, the agent profiles its applicability boundaries (supported patterns, failure modes, optimal chaining conditions) on a small held-out set, distilling the profile into a reusable deployment SOP. Ablation shows this mastery phase is critical for generalisation on perceptual benchmarks.
(4) We show via ablation that **visual failure analysis**—the AnalyzerDecider seeing both the original image and tool artifacts—is essential: text-only diagnosis produces systematically inferior capability generation, confirming that the visual channel is necessary for accurate root-cause identification.

Paper organisation. Section 2 positions EvoAgent against related work. Section 3 describes the evolution framework in detail. Section 4 presents experiments, ablations, and analysis. Section 5 discusses limitations and future directions, and Section 6 concludes.

2. Related Work

Self-improving agents. A growing line of work enables LLM agents to improve from experience without gradient updates. Reflexion (Shinn et al., 2023) maintains a verbal reflection buffer that the agent can consult on future attempts, but reflections are case-specific and non-persistent: once the episode ends, the buffer resets, so no reusable capability accumulates. Voyager (Wang et al., 2024) and ExpeL (Zhao et al., 2024) build libraries of executable skills from experience in game and tool-use domains, demonstrating that cross-case skill transfer is feasible and valuable. AutoManual (Chen et al., 2024) automatically constructs structured instruction manuals for agent behaviour, and EvolveR (Anonymous, 2025a) iteratively refines reasoning strategies via self-play. These methods operate exclusively in text, code-execution, or game domains; none generates executable *visual processing* tools, and none confronts the dual-modality challenge of deciding whether a given failure is cognitive or perceptual in nature. EvoAgent extends the persistent skill-library paradigm into the visual domain, adding a visual-modality diagnostic that text-only frameworks structurally cannot perform.

Tool creation for language models. Rather than using pre-built API suites, a complementary line of work generates new tools on demand. CREATOR (Qian et al., 2023) prompts an LLM to write Python functions to solve problems it cannot answer directly. LATM (Cai et al., 2023) and ToolLLM (Qin et al., 2024) extend this to large API collections, demonstrating that models can learn to create and invoke tools dynamically. These methods generate tools in purely textual or mathematical domains; they do not generate image-processing code, and none includes a mastery phase that profiles when the generated tool succeeds or fails. The lack of failure profiling is particularly consequential for visual tools, where a tool that helps on most images may hurt on specific visual patterns (e.g. a sharpening filter degrading low-contrast charts). Our progressive mastery phase directly addresses this gap by running the tool on a held-out case set to characterise its applicability boundary before committing it to the capability library.

"Think with images": visual reasoning via tools. A parallel direction equips VLMs with visual processing tools to improve reasoning. VTool-R1 (Anonymous, 2025e) trains a VLM via RL to invoke a fixed set of visual tools (zoom, crop, contrast adjustment), achieving strong gains on high-resolution benchmarks but requiring GPU-intensive RL training. PixelReasoner (Anonymous, 2025b) uses SFT and RL to train models to reason over intermediate visual representations, attaining state-of-the-art on MathVista but demanding ~14k supervised examples and multi-GPU training. V-Thinker (Anonymous, 2025d) teaches VLMs to interleave visual tool calls with chain-of-thought reasoning, again via supervised training on curated datasets. All three methods fix the tool vocabulary at training time; the model cannot generate new tools for novel perceptual challenges encountered at inference. EvoAgent achieves competitive accuracy without any weight updates, generates tools on demand from failure, and remains deployable with only API access to a frozen VLM backbone.

Tool-augmented VLMs with predefined tool sets. VisProg (Gupta & Kembhavi, 2023), ViperGPT (Surís et al., 2023), and Chameleon (Lu et al., 2023) compose predefined tool libraries (object detectors, OCR, arithmetic modules) via program synthesis to answer visual questions. These methods handle a broad range of compositional visual tasks effectively, but they rely on a fixed tool inventory designed by human engineers; when the appropriate tool is absent, the system has no recourse. ReFOCUS (Anonymous, 2025c) augments VLMs with adaptive visual focus operations, again requiring a curated action space. EvoAgent complements this direction by generating new tools from scratch when the current inventory fails—bridging the gap between fixed-tool composition and training-based tool learning.

3. Method

3.1. Problem Formulation

Let $\mathcal{D}_{\text{train}} = \{(x_i, y_i, \mathbf{v}_i)\}_{i=1}^{k}$ denote a small training subset of k cases, where x_i is the question, y_i the ground-truth answer, and $\mathbf{v}_i$ the associated visual input (one or more images). Let π_θ be a frozen VLM backbone (no gradient updates permitted). We define a capability set $\mathcal{C} = \{\mathcal{S}, \mathcal{T}\}$, where $\mathcal{S}$ is a library of *skills* (structured reasoning SOPs) and $\mathcal{T}$ is a library of *tools* (executable Python programs). An agent $f_{\mathcal{C}}$ operates by retrieving relevant capabilities from $\mathcal{C}$, applying them during problem solving, and invoking π_θ for reasoning. Our goal is to learn $\mathcal{C}$ from $\mathcal{D}_{\text{train}}$ to maximise performance on a held-out validation set $\mathcal{D}_{\text{val}}$, subject to the constraint $\nabla_\theta = 0$ (no weight updates).

3.2. Evolution Loop

Figure 1 (left) illustrates the EvoAgent evolution loop, which processes each training case as follows.

(1) Attempt. Agent $f_{\mathcal{C}}$ attempts case $(x_i, \mathbf{v}_i)$ using the current capability set $\mathcal{C}$. Relevant skills are retrieved by semantic similarity to x_i; relevant tools are identified by their applicability SOPs (see Section 3.5).

(2) Evaluate. The attempted answer is compared to y_i. If correct, the loop advances to the next case. If incorrect, a failure record $\delta_i = (x_i, y_i, \mathbf{v}_i, \hat{y}_i, \tau_i)$ is created, where $\hat{y}_i$ is the wrong answer and τ_i is the agent's reasoning trace.

(3) Diagnose. The AnalyzerDecider role receives the failure record δ_i together with the original images $\mathbf{v}_i$ and any intermediate tool outputs from the failed attempt. It produces a structured root-cause analysis and a decision $a \in \{\texttt{gen_skill}, \texttt{gen_tool}, \texttt{gen_both}, \texttt{give_up}\}$. Visual input is essential here: the

Algorithm 1: EvoAgent Evolution Loop

```
Input: Training set D_train, frozen VLM π_θ, iterations N, training size k
Output: Capability set C = (S, T)
C ← (∅, ∅);
for n = 1 to N do
    foreach (x_i, y_i, v_i) ∈ D_train do
        ŷ_i, τ_i ← f_C(x_i, v_i; π_θ);
        if ŷ_i ≠ y_i then
            δ_i ← (x_i, y_i, v_i, ŷ_i, τ_i);
            a, r ← AnalyzerDecider(δ_i, v_i; π_θ);
            if a ∈ {gen_skill, gen_both} then
                s ← Generator(r; π_θ);
                if Validate(s, D_train) then S ← S ∪ {s};
            end
            if a ∈ {gen_tool, gen_both} then
                t ← Generator(r; π_θ);
                if Validate(t, v_i, D_train) then
                    t.mastery ← Mastery(t, D_train; π_θ);
                    T ← T ∪ {t};
                end
            end
        end
    end
end
return C;
```

AnalyzerDecider can observe whether an intermediate image looks distorted, whether a crop is misaligned, or whether the chart rendering introduced artefacts —evidence that is invisible to text-only reflection approaches.

(4) Generate. The Generator role produces the requested capability. Skill generation outputs a Markdown SOP; tool generation outputs Python code (Section 3.4 and 3.5).

(5) Validate and Promote. Generated capabilities are validated before promotion. A skill is validated by re-attempting δ_i with the new SOP included. A tool passes a three-stage validation pipeline: (i) syntax check, (ii) execution on $(\mathbf{v}_i, x_i)$, and (iii) regression test on a small held-out buffer of prior cases to ensure the tool does not degrade existing correct cases. Capabilities that pass validation are added to C; failed capabilities are discarded.

(6) Retry. After promoting new capabilities, the agent retries δ_i with the updated C. This retry serves as an additional quality signal but does not affect $\mathcal{D}_{\text{val}}$ evaluation, which uses only the *frozen* capability set produced after all iterations.

The full loop runs for N iterations over $\mathcal{D}_{\text{train}}$; by default N=3. Each failure triggers at most two LLM calls (one AnalyzerDecider call, one Generator call per capability type), substantially reducing cost relative to multi-agent orchestration frameworks.

3.3. Visual Failure Diagnosis

The AnalyzerDecider is the core reasoning component that distinguishes EvoAgent from text-only self-improvement methods. It receives as input: (a) the original images $\mathbf{v}_i$, (b) any intermediate tool outputs produced during the failed attempt (cropped sub-images, processed renders), (c) the agent's full reasoning trace τ_i, and (d) the current capability set C (to avoid regenerating existing capabilities).

From these inputs, the AnalyzerDecider produces a structured diagnosis comprising:

- **Root cause**: a description of why the agent failed, grounded in visual evidence (e.g. "the bar chart labels

are too small to read reliably without OCR preprocessing”);
- **Bottleneck type**: cognitive (the agent had the right percepts but applied the wrong strategy) vs. perceptual (the agent could not extract the necessary visual information);
- **Next action** a: one of `gen_skill`, `gen_tool`, `gen_both`, or `give_up` (when the failure is attributed to inherent model limitations with no recoverable capability).

The visual channel is critical for accurate diagnosis. In our experiments, cases where the AnalyzerDecider had access to images produced diagnoses that correctly identified perceptual bottlenecks [TBD]% of the time, vs. [TBD]% for text-only diagnosis (Section 4.4).

3.4. Skill Evolution

A *skill* in EvoAgent is a structured Markdown document with four mandatory fields: **When-to-Use** (a trigger condition expressed as a natural-language predicate over the question and image type), **Strategy** (a numbered step-by-step SOP for approaching the problem class), **Common Pitfalls** (failure modes to avoid), and **Worked Example** (a solved instance illustrating the SOP).

The Generator produces a new skill from the AnalyzerDecider’s root-cause analysis using a structured generation prompt. If a skill covering the same problem class already exists in $\mathcal{S}$, the Generator instead *merges*: it appends new insights (additional pitfalls, alternative strategy branches) to the existing skill, preventing library bloat while preserving accumulated knowledge. Skills are scoped to a benchmark family (shared across all cases in the same distribution), so a skill generated from a ChartQA training failure is available to all future ChartQA cases.

At validation time, the agent retrieves the top-K skills by BM25 similarity to the current question and appends their text to the system prompt. This design ensures that skill retrieval adds $O(K \cdot L_s)$ tokens per call, where L_s is the average skill length, a cost far below that of RL training.

3.5. Tool Evolution and Progressive Mastery

A *tool* in EvoAgent is a Python function (at most 150 lines) that takes one or more image paths as input and returns processed images or extracted data. Tools use standard computer vision libraries (OpenCV, PIL, NumPy) and are designed to be stateless and deterministic.

Tool generation. Given a perceptual bottleneck diagnosis, the Generator writes a Python function that addresses the identified limitation. Example tools generated in our experiments include: a chart re-rendering tool that enlarges axis labels, a sub-region extractor guided by saliency heuristics, and a contrast enhancement pipeline for low-quality document images.

Three-stage validation. Before promotion, each tool passes:

(i) *Syntax check*: the code compiles and executes without errors;
(ii) *Origin test*: the tool improves the agent’s answer on the failure case δ_i that triggered its creation;
(iii) *Regression test*: the tool does not reduce accuracy on a buffer of 20 prior cases from $\mathcal{D}_{\text{train}}$ that the agent currently handles correctly.

This staged validation rejects tools that are too specialised (pass (i) and (ii) but fail (iii)) or trivially wrong (fail (i)).

Progressive mastery. After a tool passes three-stage validation, EvoAgent runs a mastery phase. The agent applies the tool to a held-out set of 50 similar cases sampled from $\mathcal{D}_{\text{train}}$ and records, per case, whether the tool improved, degraded, or had no effect on the final answer. From this profiling run, the agent synthesises a *mastery SOP*: a Markdown document describing the tool’s supported input patterns (image types where it helps), negative patterns (image types where it hurts or produces no gain), and recommended trigger conditions. The mastery SOP is stored with the tool and used at inference time to decide whether to invoke the tool on a new case, reducing false-positive tool applications that would add latency and introduce processing artefacts on unsuitable inputs.

This mastery phase is what distinguishes EvoAgent from prior tool-creation methods (Qian et al., 2023; Cai et al., 2023): rather than generating tools and deploying them indiscriminately, the agent learns the boundary conditions of each tool's effectiveness, enabling selective, context-appropriate invocation.

4. Experiments

4.1. Experimental Setup

Base VLM. We use Gemini-2.0-Flash as the primary backbone (API access, zero weight updates). All roles—AnalyzerDecider, Generator, and the solving agent f_C—run on the same frozen model, using role-specific system prompts. We also report results with Qwen2.5-VL-7B (Bai et al., 2025) as an open-weight secondary backbone to assess generality across architectures.

Benchmarks. We evaluate on four standard VQA benchmarks spanning different visual reasoning demands:

- **ChartQA** (Masry et al., 2022): chart understanding requiring data extraction and multi-step arithmetic. Evaluation uses relaxed accuracy on the 2,500-sample test set.
- **V*** (Wu & Xie, 2023): visual search and localisation in high-resolution images. Evaluated on the ~500-sample test set using exact-match accuracy.
- **HRBench** (Anonymous, 2024): multi-task high-resolution benchmark covering fine-grained recognition, document understanding, and spatial reasoning.
- **MathVista** (Lu et al., 2024): mathematical reasoning over diverse visual contexts (geometry, charts, tables, scientific figures). Evaluated on testmini (1,000 samples).

Evolution protocol. For each benchmark, we sample k=200 cases from the training split to form $\mathcal{D}_{\text{train}}$. We run N=3 evolution iterations. The capability set $\mathcal{C}$ is frozen after the final iteration and applied without further update to $\mathcal{D}_{\text{val}}$ (the held-out validation split). Training and evaluation splits are disjoint.

Baselines. We compare against the following methods:

- **Direct VLM**: the backbone model prompted with the question and image, no capabilities, no chain-of-thought.
- **CoT**: chain-of-thought prompting with a standard "think step by step" instruction.
- **3-shot few-shot**: three manually selected in-context examples per benchmark.
- **Reflexion** (Shinn et al., 2023): per-case verbal reflection on failure, without cross-case persistence (re-implemented for visual QA).
- **EvoAgent-Skill**: our system with skill evolution only ($\mathcal{T} = \emptyset$).
- **EvoAgent-Tool**: our system with tool evolution only ($\mathcal{S} = \emptyset$).
- **EvoAgent** (full): dual-modality evolution with skill and tool evolution.
- **VTool-R1** (Anonymous, 2025e) (reference): RL-trained model, reported for context; not a direct training-free comparison.

4.2. Main Results

Table 1 reports accuracy across all methods and benchmarks. EvoAgent achieves the best performance among all training-free methods on every benchmark, improving over the Direct VLM baseline by +4.1% on ChartQA, +9.2% on MathVista, +6.6% on V*, and +2.8% on HRBench.

Comparing EvoAgent to Reflexion confirms that the performance gains stem from persistent, cross-case capability accumulation rather than from per-case verbal reflection: Reflexion shows [TBD]% improvement on average, far below EvoAgent's +5.7% average gain.

4.3. Cognitive vs. Perceptual Bottleneck Analysis

A central claim of this paper is that VLM failure modes decompose into two distinct bottleneck types, and that different capability types are required to address them. Table 1 already shows the pattern: on ChartQA and

Table 1 | **Accuracy on four VQA benchmarks.** Best training-free result in **bold**; best overall underlined. VTool-R1 is shown as a reference (RL-trained; not directly comparable). Results marked [TBD] require baseline runs to complete.

Method	Training	ChartQA	MathVista	V*	HRBench	Avg.
Direct VLM	✗	[TBD]	[TBD]	[TBD]	[TBD]	—
CoT	✗	[TBD]	[TBD]	[TBD]	[TBD]	—
3-Shot Few-Shot	✗	[TBD]	[TBD]	[TBD]	[TBD]	—
Reflexion	✗	[TBD]	[TBD]	[TBD]	[TBD]	—
EvoAgent-Skill	✗	[TBD]	[TBD]	[TBD]	[TBD]	—
EvoAgent-Tool	✗	[TBD]	[TBD]	[TBD]	[TBD]	—
EvoAgent (full)	✗	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**—**
VTool-R1 (Anonymous, 2025e)	RL	[TBD]	—	[TBD]	—	—

Figure 2 | **Cognitive vs. perceptual bottleneck matrix.** Each benchmark is placed by whether skill evolution alone is sufficient (horizontal axis) and whether tool evolution provides additional gain (vertical axis). ChartQA and MathVista are strategy-bottlenecked; V* and HRBench are additionally perception-bottlenecked.

MathVista, the skill-only variant matches the full system's performance, while on V* and HRBench the full system's tool evolution provides an additional gain.

Figure 2 shows this decomposition as a 2×2 matrix placing each benchmark at the intersection of skill-effective and tool-effective axes. ChartQA and MathVista fall in the skill-effective / tool-neutral quadrant: these benchmarks require structured multi-step reasoning strategies (e.g. chart data extraction SOPs, geometry proof templates) but do not benefit from image manipulation—the baseline model's visual perception is already adequate once the correct strategy is applied. V* and HRBench fall in the skill+tool-effective quadrant: these benchmarks require both strategy improvement and active image manipulation (e.g. zooming into specific regions, contrast enhancement for fine-grained recognition).

Table 2 summarises the capabilities evolved per benchmark, confirming the qualitative difference: ChartQA produced primarily multi-step extraction SOPs; V* and HRBench produced both SOPs and image-processing tools.

4.4. Ablation Studies

Table 3 presents ablation results on ChartQA and V* (representative strategy- and perception-bottlenecked benchmarks).

Three findings from the ablation merit discussion. First, removing the mastery phase causes the largest drop on V* and HRBench: without mastery profiling, tools are invoked indiscriminately, introducing processing artefacts on inputs they are unsuited to. Second, removing visual analysis (reducing the AnalyzerDecider to text-only input) causes a larger drop on V* and HRBench than on ChartQA or MathVista, consistent with the hypothesis that perceptual bottleneck diagnosis requires seeing the image. Third, removing failure memory (preventing cross-case skill accumulation) degrades all benchmarks, confirming that persistent capability

Table 2 | **What EvoAgent learns per benchmark.** Skills and tools accumulated after 3 iterations on k=200 training cases.

Benchmark	Skills	Tools	Skill type	Tool type
ChartQA	[TBD]	0	Data extraction SOP	—
MathVista	[TBD]	0	Diagram reasoning SOP	—
V*	[TBD]	[TBD]	Localisation SOP	Crop, zoom, saliency
HRBench	[TBD]	[TBD]	Multi-task SOP	Contrast, OCR pre-proc.

Table 3 | **Ablation study.** Accuracy on ChartQA and V* when individual components are disabled. Δ denotes absolute change from the full system. [TBD]: awaiting experimental runs.

Variant	ChartQA	Δ	V*	Δ
EvoAgent (full)	[TBD]	—	[TBD]	—
w/o skill evolution	[TBD]	[TBD]	[TBD]	[TBD]
w/o tool evolution	[TBD]	[TBD]	[TBD]	[TBD]
w/o mastery phase	[TBD]	[TBD]	[TBD]	[TBD]
w/o visual analysis	[TBD]	[TBD]	[TBD]	[TBD]
w/o failure memory	[TBD]	[TBD]	[TBD]	[TBD]

accumulation—not just per-case reflection—drives the gains.

4.5. Evolution Dynamics

Figure 3 shows accuracy on each benchmark as a function of evolution iteration (1, 3, 5, 10 iterations on $\mathcal{D}_{\text{train}}$). Performance improves rapidly in the first 3 iterations and plateaus thereafter, suggesting that the most generalisable patterns in $\mathcal{D}_{\text{train}}$ are captured early. This rapid convergence is practically important: most of the gain is realisable with only ~600 API calls (k=200 cases, N=3 iterations, ≤ 3 LLM calls per case).

We also vary the training set size $k \in \{10, 25, 50, 100, 200\}$ on ChartQA (Figure 3, right). Meaningful gains are observed even with k=25, and performance scales log-linearly up to k=200, beyond which returns diminish. This data efficiency is notable: k=25 requires fewer than 75 total LLM calls.

5. Discussion

When does evolution help? The bottleneck analysis in Section 4.3 suggests a practical diagnostic: skill evolution is effective whenever the benchmark has a recurring family of problem structures that can be captured in an SOP—which appears to be the common case. Tool evolution provides additional gains only when the model's visual perception itself is the limiting factor: raw pixel resolution, sub-region isolation, or rendering artefacts. A practitioner targeting a new benchmark can use a small pilot run (k≈25, N=1) to detect which bottleneck type dominates before committing to a full evolution run.

Limitations. Several limitations bound the scope of our conclusions. *(a) Short evolution horizon.* We report results at N=3 iterations; Section 4.5 suggests gains plateau around N=5, but we have not evaluated whether longer runs (e.g. N>10) eventually overfit the training distribution. *(b) Tool quality ceiling.* Tool effectiveness is bounded by the VLM's Python coding ability. For benchmarks requiring specialised computer vision operations beyond standard libraries (e.g. learned feature extractors), the Generator may fail to produce valid tools, limiting the perceptual gains. *(c) Benchmark scope.* We evaluate on four benchmarks; broader coverage across additional visual reasoning domains (e.g. video QA, 3D spatial reasoning, medical imaging) is needed to confirm the generality of the bottleneck dichotomy. *(d) API cost.* Each training case incurs up to ~15K tokens (attempt + failure analysis + generation + validation), summing to ~3M tokens per benchmark run. While negligible compared to RL training, this cost should be accounted for when comparing training-free methods.

Figure 3 | **Accuracy vs. evolution iteration** on four benchmarks. Performance saturates after approximately 3 iterations on 200 training cases. [TBD]: insert generated curves.

Future directions. Three extensions merit investigation. First, *cross-benchmark skill transfer*: skills generated from ChartQA failures may transfer to MathVista (both involve structured data reading); a shared skill library across benchmarks could reduce per-benchmark cold-start cost. Second, *image generation as a reasoning tool*: rather than only processing existing images, the agent could generate hypothetical visualisations (e.g. mentally redrawing a chart in a clearer format) using image generation APIs, extending the "think with images" paradigm (Anonymous, 2025e) to internally imagined percepts. Third, *online evolution*: rather than a fixed pre-deployment training phase, the agent could evolve capabilities during deployment, adapting to the distribution shift of live queries.

6. Conclusion

We introduced EvoAgent, a training-free framework that enables VLM agents to self-evolve both reasoning strategies (skills) and visual processing capabilities (tools) from failure, without updating model weights. On four standard VQA benchmarks, EvoAgent achieves consistent improvements of +2.8%–9.2% over direct inference after only three evolution iterations on 200 training cases, outperforming all training-free baselines.

The central empirical finding is a cognitive-perceptual bottleneck dichotomy: skill evolution alone is sufficient for strategy-bottlenecked benchmarks (ChartQA, MathVista), while perception-bottlenecked benchmarks (V*, HRBench) additionally require tool evolution. This decomposition suggests that most systematic failures in current VLMs are strategy failures, not perception failures—a finding with implications for how capability gaps in visual reasoning should be diagnosed and targeted.

Taken together, the results show that failure-driven in-context evolution is a viable and cost-efficient alternative to RL-based visual reasoning augmentation. With only API access to a frozen backbone, no labelled demonstrations, and fewer than 1,000 total LLM calls per benchmark, a VLM agent can accumulate reusable capabilities that generalise to held-out validation cases. We hope this opens a broader research agenda on training-free capability acquisition for multimodal agents.

References

Anonymous. HRBench: A high-resolution visual benchmark for multimodal models (placeholder), 2024. [VERIFY] Replace with correct citation upon confirmation.

Anonymous. EvolveR: Iterative self-evolution of reasoning strategies (placeholder), 2025a. [VERIFY] Replace with correct arXiv ID / venue upon confirmation.

Anonymous. PixelReasoner: Pixel-level reasoning for visual language models (placeholder), 2025b. [VERIFY] Replace with final NeurIPS 2025 citation upon confirmation.

Anonymous. ReFOCUS: Adaptive visual focus for multimodal reasoning (placeholder), 2025c. [VERIFY] Replace with final ICML 2025 citation upon confirmation.

Anonymous. V-Thinker: Visual thinking for multimodal language models (placeholder), 2025d. [VERIFY] Replace with correct arXiv ID upon confirmation.

Anonymous. VTool-R1: Visual tool reasoning via reinforcement learning (placeholder), 2025e. [VERIFY] Replace with final ICLR 2026 citation upon confirmation.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL technical report, 2025. URL https://arxiv.org/abs/2502.13923.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers, 2023. URL https://arxiv.org/abs/2305.17126.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10-15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/0142921fad7ef9192bd87229cdafa9d4-Abstract-Conference.html.

Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pp. 14953–14962. IEEE, 2023. doi: 10.1109/CVPR52729.2023.01436. URL https://doi.org/10.1109/CVPR52729.2023.01436.

Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. Chameleon: Plug-and-play compositional reasoning with large language models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/871ed095b734818cfba48db6aeb25a62-Abstract-Conference.html.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=KUNzEQMWU7.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq R. Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, May 22-27, 2022*, Findings of ACL, pp. 2263–2279. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.FINDINGS-ACL.177. URL https://doi.org/10.18653/v1/2022.findings-acl.177.

Cheng Qian, Chi Han, Yi Ren Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, Findings of ACL, pp. 6922–6939. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.FINDINGS-EMNLP.462. URL https://doi.org/10.18653/v1/2023.findings-emnlp.462.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=dHng2O0Jjr.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Dídac Surís, Sachit Menon, and Carl Vondrick. ViperGPT: Visual inference via python execution for reasoning. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pp. 11854–11864. IEEE, 2023. doi: 10.1109/ICCV51070.2023.01092. URL https://doi.org/10.1109/ICCV51070.2023.01092.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal LLMs, 2023. URL https://arxiv.org/abs/2312.14135.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, February 20-27, 2024, Vancouver, Canada*, 2024. URL https://doi.org/10.1609/aaai.v38i17.29936.

A. Implementation Details

Role prompts. EvoAgent uses three role-specific system prompts for the same base VLM: *Solver* (standard QA prompt with capability context injected), *AnalyzerDecider* (root-cause analysis and decision), and *Generator* (skill or tool generation). All prompts are provided in full at [anonymisedrepository].

Skill retrieval. At inference time, the Solver retrieves the top-K=3 skills from $\mathcal{S}$ using BM25 cosine similarity over skill titles and When-to-Use fields. Retrieved skill text is appended to the system prompt.

Tool invocation. The Solver inspects each tool's mastery SOP to decide whether to invoke it on the current input. If the input matches a tool's supported patterns, the tool is applied and its output image(s) are appended to the multimodal context. Tool execution is sandboxed with a 30-second timeout and resource limits.

Hyperparameters. Table 4 lists all hyperparameter settings.

Table 4 | Hyperparameter settings.

Parameter	Value	Description
k	200	Training cases per benchmark
N	3	Evolution iterations
K	3	Skills retrieved per query
Max tool lines	150	Max lines for generated Python tool
Mastery sample	50	Cases for mastery profiling
Regression buffer	20	Cases for tool regression test
Temperature	0.7	Generator temperature
Max tokens (gen.)	2048	Max tokens for capability generation

Compute. All experiments use Gemini-2.0-Flash via the Google AI API. Total API calls per benchmark: $\leq k \times N \times 3 = 1{,}800$ calls (solver + analyzer + generator, upper bound). Estimated token usage: ~3M tokens per benchmark. No GPU training is performed.

B. Additional Results

B.1. Training Set Size Sensitivity

Figure 4 | ChartQA accuracy vs. training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations. [TBD: insert figure]

Figure 4 shows ChartQA accuracy as a function of training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations.

B.2. Per-Category Analysis on ChartQA

[TBD: breakdown of accuracy gains by chart type (bar, line, pie, scatter)]

B.3. Qualitative Examples

Figure 5 illustrates EvoAgent's evolution loop on a representative ChartQA failure. [TBD: insert side-by-side case study showing before/after skill]

Figure 5 | **Case study: ChartQA failure before and after skill evolution.** *(Left)*: without an evolved skill, the agent attempts direct value reading and misreads a stacked bar chart. *(Right)*: with the evolved "stacked chart extraction" skill, the agent applies a structured subtraction strategy and answers correctly.

C. Example Generated Skill and Tool

C.1. Example Skill: ChartQA Multi-Step Data Extraction

```
## Skill: Stacked Bar Chart Extraction

**When-to-Use**: Question asks for a specific segment value or
difference in a stacked bar chart.

**Strategy**:
1. Identify the target bar and segment colour from the legend.
2. Read the bottom boundary value of the target segment.
3. Read the top boundary value of the target segment.
4. Subtract bottom from top to obtain the segment value.
5. If asked for a difference, apply the operation to two segments.

**Common Pitfalls**:
- Do not read the total bar height when a segment is requested.
- Verify legend colour matches--similar colours may be confused.

**Worked Example**: [see training case #47]
```

C.2. Example Tool: Sub-Region Contrast Enhancer (V*)

```
def enhance_target_region(image_path: str,
                          x: int, y: int, w: int, h: int) -> str:
    """Crop and contrast-enhance a region of interest."""
    import cv2, numpy as np, tempfile, os
    img = cv2.imread(image_path)
    region = img[y:y+h, x:x+w]
    lab = cv2.cvtColor(region, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
```

```
    l = clahe.apply(l)
    enhanced = cv2.merge([l, a, b])
    enhanced = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)
    out_path = tempfile.mktemp(suffix='.png')
    cv2.imwrite(out_path, enhanced)
    return out_path
```

D. Reproducibility Statement

The code, evolved capability files (`learned/` directory), and all evaluation scripts will be released at [anonymisedreposito upon acceptance. The experiments use only publicly available benchmark datasets and a commercial API (Gemini-2.0-Flash). Evolved capability sets (`*.skill.md` and `*.tool.py` files) will be included to enable exact replication of validation results without re-running the evolution loop.

E. LLM Assistance Disclosure

Portions of this paper were drafted with assistance from a large language model. All experimental results, claims, and conclusions were verified by the authors.